Documentación técnica: predictor de fallas HDD

1. Propósito y alcance

Este programa prepara historiales SMART de discos HDD, entrena y evalúa un modelo de supervivencia Cox Elastic Net, y permite estimar la mediana modelada del tiempo hasta falla para un disco. Está pensado para análisis exploratorio y priorización con revisión humana. No es una alarma calibrada ni un sistema de reemplazo automático.

La predicción depende de que las definiciones y unidades de SMART coincidan con las del conjunto de entrenamiento. La salida no es una fecha garantizada ni una probabilidad de falla.

2. Arquitectura y flujo de datos

Diagrama de flujo (ver file mermaid-diagram-2026-10-20-233904.png)

baseline.py define las siete columnas predictoras y convierte observaciones repetidas en duraciones hasta la fecha de falla o censura. cox_elastic_net.py separa los seriales completos entre pliegues para evitar que observaciones de un mismo HDD queden en entrenamiento y prueba. El preprocesamiento se ajusta solo en cada partición de entrenamiento. Tras la evaluación, se seleccionan hiperparámetros mediante Validación Cruzada Agrupada y se reajusta un modelo final con todos los datos. predecir_falla_hdd.py carga ese modelo y sus transformadores para inferencia.

3. Contenido del programa

- baseline.py: columnas requeridas, construcción de duración/evento y utilidades del baseline.
- cox_elastic_net.py: evaluación agrupada anidada, selección de hiperparámetros, ajuste final y exportación de resultados.
- predecir_falla_hdd.py: inferencia para una fila de atributos SMART.
- requirements.txt: versiones fijadas de bibliotecas Python.
- failed_drive_histories.csv: datos de ejemplo presentes en este paquete.
- cox_elastic_net_output/: si existe, resultados/modelo previamente generados.

Un CSV propio debe tener las columnas date, serial_number, failure y las siete columnas SMART normalizadas: smart_5_normalized, smart_9_normalized, smart_187_normalized, smart_188_normalized, smart_197_normalized, smart_198_normalized, smart_199_normalized. La columna failure debe codificar 1 para falla observada y 0 para censura. Cada HDD debe tener varias observaciones anteriores a la fecha terminal para generar duraciones positivas.

4. Instalación

Se recomienda Python 3 de 64 bits (Se probo usando Python 3.14.4 x64). Las dependencias están fijadas en requirements.txt; la compatibilidad con otros intérpretes/plataformas puede depender de la disponibilidad de las dependencias en esas versiones.

Windows (PowerShell)

```
py -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

pip install -r requirements.txt
```

macOS/Linux

```
python3 -m venv .venv

source .venv/bin/activate

python -m pip install --upgrade pip

pip install -r requirements.txt
```

5. Ejemplos de uso

Entrenar y evaluar

Desde la carpeta cox_hdd_portable:

```
python cox_elastic_net.py --input failed_drive_histories.csv
```

Ejemplo con rutas y parámetros explícitos:

```
python cox_elastic_net.py --input failed_drive_histories.csv --output-dir resultados --seed 2026 --bootstraps 2000
```

En Windows PowerShell:

```
python .\cox_elastic_net.py --input .\failed_drive_histories.csv --output-dir .\resultados --seed 2026 --bootstraps 2000
```

La carpeta de salida contiene cox_elastic_net_models.joblib (modelo final, transformadores y modelos por pliegue), experiment_results.json (métricas y configuración), inner_trials.csv (búsqueda de hiperparámetros) y analysis_dataset.csv (tabla derivada de duración/evento).

Predecir

Después de entrenar, pasar los siete atributos SMART normalizados:

ejemplo:
python predecir_falla_hdd.py --smart_5_normalized 100 --smart_9_normalized 90 --smart_187_normalized 100 --smart_188_normalized 100 --smart_197_normalized 100 --smart_198_normalized 100 --smart_199_normalized 200

Para usar otro artefacto, añada --model ruta/al/cox_elastic_net_models.joblib. La predicción falla de forma explícita si falta un atributo que el modelo final necesita.

6. Opciones de configuración

Opciones de línea de comandos del entrenamiento

Opción	**Predeterminado**	**Qué modifica**
--input	Obligatoria	Ruta del CSV fuente. Cambia los datos utilizados para construir y ajustar el modelo.
--output-dir	cox_elastic_net_output junto al script	Directorio de resultados y artefactos; se crea si no existe.
--seed	2026	Semilla para particiones agrupadas y bootstrap; cambiarla cambia particiones/muestreo y puede cambiar métricas e hiperparámetros seleccionados.
--bootstraps	2000	Número de réplicas para intervalos bootstrap. Debe ser al menos 100; más réplicas estabilizan la estimación Monte Carlo, pero

Opción	Predeterminado	Qué modifica
		no añaden datos independientes.

Opciones de inferencia

Opción	Predeterminado	Qué modifica
--model	cox_elastic_net_output/cox_elastic_net_models.joblib junto al script	Artefacto cargado para transformar y predecir.
--smart_5_normalized, --smart_187_normalized, --smart_188_normalized, --smart_197_normalized, --smart_198_normalized, --smart_199_normalized, --smart_9_normalized	Ninguno	Entrada numérica por atributo. Deben entregarse todos los que el modelo final conserva; faltantes causan rechazo.

Parámetros definidos en el código

No tienen opción CLI en esta versión. Modificarlos requiere cambiar el código y volver a evaluar:

Constante	Valor actual	Efecto
PENALIZERS	0.01, 0.1, 1.0	Candidatos para fuerza de penalización durante selección.
L1_RATIOS	0.25, 0.75	Candidatos para mezcla L1/L2 de Elastic Net.
OUTER_FOLDS	5	Número máximo de particiones externas para evaluación.
INNER_FOLDS	3	Particiones internas para selección de hiperparámetros y ajuste final.

Constante	Valor actual	Efecto
FEATURES	Siete SMART normalizados indicados arriba	Predictores usados al construir y ajustar el modelo. Cambiar columnas exige datos compatibles y reentrenamiento.
Imputación/escalado	Mediana / StandardScaler	Reglas de preprocesamiento; se ajustan por pliegue y se guardan con el modelo.

Los pliegues se agrupan por serial y pueden reducirse si el número de seriales/eventos es insuficiente. Los resultados pueden cambiar con datos distintos, semilla, versiones de bibliotecas o parámetros.

7. Desempeño y limitaciones conocidas

En los datos documentados (100 seriales, 7.601 observaciones; 50 HDD con falla y 50 censurados), validación anidada de cinco pliegues agrupados por serial dio C-index macro 0.6408 para Cox baseline e Elastic Net. Para Elastic Net el IC bootstrap 95% fue [0.5755, 0.7042]; la diferencia pareada frente al baseline fue 0.0000 en esa ejecución. Esto es evidencia limitada de ordenamiento relativo en trayectorias con fallas.

Limitaciones:

- El C-index del modelo es bajo.
- No se evaluaron Brier score, calibración de riesgo a horizontes, sensibilidad/especificidad o beneficio neto.
- La evaluación puntúa 50 seriales con eventos. No valida el comportamiento en otros fabricantes, firmware, población o periodo.
- La mediana no trae intervalo de predicción individual y puede no ser finita si la supervivencia no cruza 0.5.
- Se requieren fechas/eventos fiables, varias observaciones por HDD y suficiente cantidad de seriales con eventos para validación agrupada.
- El archivo de datos contiene seriales. Restrinja acceso, evite incluir datos en repositorios públicos y defina retención/borrado.

8. Mantenimiento y ciclo de vida

El modelo no se actualiza solo. Mantener el programa requiere conservar el CSV/esquema, dependencias compatibles y el artefacto generado. Con el tiempo pueden dejar de ser válidos los supuestos o dejar de funcionar partes concretas:

- **Cambios de datos/esquema:** renombrar columnas, cambiar unidades SMART, firmware, frecuencia de muestreo o codificación de failure puede provocar errores de carga o, peor, entradas semánticamente incorrectas. Actualizar validaciones y definiciones; reconstruir y revisar el dataset.
- **Deriva temporal/de población:** desgaste, políticas de reemplazo, nuevos HDD, sensores o firmware pueden degradar el ajuste aunque el código corra. Requiere validación temporal/externa y, si el rendimiento/calibración cae o la población cambia, reentrenamiento con datos recientes y etiquetas maduras.
- **Cambios de censura y proceso de registro:** nuevas reglas para declarar falla o fin de observación cambian el objetivo. Revisar etiquetas y volver a construir/evaluar antes de reentrenar.
- **Cambios de dependencias o Python:** versiones incompatibles pueden impedir cargar .joblib, cambiar resultados o romper APIs. Mantener el entorno fijado; al actualizar dependencias, regenerar entorno, comprobar compatibilidad y comparar evaluación antes de publicar.
- **Cambios de features/preprocesamiento/modelo:** requieren reentrenar, guardar juntos modelo y transformadores y repetir validación agrupada, calibración y análisis de grupos. No reutilizar un artefacto antiguo con columnas nuevas.
- **Artefacto perdido o dañado:** deja de funcionar la predicción hasta restaurar un artefacto compatible o reentrenar desde datos autorizados.

Como mínimo, tras publicar monitorear tasa de entradas rechazadas/faltantes, validez y distribución SMART, cambios de firmware/modelo, cobertura y métricas por grupo, desempeño con fallas observadas, uso humano/anulaciones, incidentes de datos, versiones del modelo y dependencias. Suspender o restringir el uso si los datos salen del soporte o cae el desempeño; revalidar antes de reanudar. No reentrenar automáticamente sin revisión de calidad, evaluación independiente y aprobación de la nueva versión.